

React Testing Library

Введение в React Testing Library

React Testing Library (RTL) — это набор утилит, которые позволяют тестировать React-компоненты без опоры на их внутреннюю реализацию. Библиотека была создана Кентом К. Доддсом как альтернатива Enzyme, с фокусом на тестирование компонентов так, как они используются конечными пользователями, а не на тестирование внутренней реализации компонентов.

Основная философия RTL заключается в том, что тесты должны максимально приближаться к тому, как пользователи взаимодействуют с приложением. Это означает, что вместо тестирования состояния компонента или его методов, вы тестируете то, что видит и с чем взаимодействует пользователь.

Установка

Для установки React Testing Library в ваш проект используйте npm или yarn:

```
npm install --save-dev @testing-library/react @testing-library/jest-dom

# или с использованием yarn
yarn add --dev @testing-library/react @testing-library/jest-dom
```

[@testing-library/jest-dom](#) предоставляет набор пользовательских матчеров Jest для DOM, которые делают ваши тесты более декларативными и легкими для чтения.

Основные концепции

Рендеринг компонентов

Функция `render` из RTL используется для рендеринга React-компонентов в виртуальный DOM для тестирования.

```
import { render, screen } from '@testing-library/react';
import MyComponent from './MyComponent';

test('отображает приветствие', () => {
  render(<MyComponent />);
  expect(screen.getByText('Привет, мир!')).toBeInTheDocument();
});
```

Запросы

RTL предоставляет несколько типов запросов для поиска элементов в DOM:

- `getBy...`: Возвращает элемент или выбрасывает исключение, если элемент не найден или найдено несколько элементов.
- `queryBy...`: Возвращает элемент или `null`, если элемент не найден. Выбрасывает исключение, если найдено несколько элементов.
- `findBy...`: Возвращает Promise, который разрешается, когда элемент найден, или отклоняется, если элемент не найден или найдено несколько элементов после таймаута.
- `getAllBy...`, `queryAllBy...`, `findAllBy...`: Версии вышеуказанных запросов, которые возвращают массив всех совпадающих элементов.

Каждый тип запроса имеет несколько вариантов, таких как `ByText`, `ByRole`, `ByLabelText`, `ByPlaceholderText`, `ByAltText`, `ByDisplayValue`, `ByTestId`.

```
// Примеры различных запросов
const button = screen.getByRole('button', { name: 'Отправить' });
const heading = screen.getByText('Заголовок');
const input = screen.getByLabelText('Имя пользователя');
const image = screen.getByAltText('Логотип');
const element = screen.getByTestId('custom-element');
```

Пользовательские события

RTL предоставляет утилиту `fireEvent` для симуляции пользовательских событий, таких как клики, ввод текста и т.д.

```
import { render, screen, fireEvent } from '@testing-library/react';
import Counter from './Counter';

test('увеличивает счетчик при клике на кнопку', () => {
  render(<Counter />);
  const button = screen.getByText('Увеличить');

  fireEvent.click(button);

  expect(screen.getByText('Счетчик: 1')).toBeInTheDocument();
});
```

userEvent

`userEvent` — это библиотека, построенная на основе `fireEvent`, которая предоставляет более реалистичную симуляцию пользовательских взаимодействий.

```
import { render, screen } from '@testing-library/react';
import userEvent from '@testing-library/user-event';
import LoginForm from './LoginForm';
```

```

test('отправляет форму с введенными данными', async () => {
  render(<LoginForm onSubmit={jest.fn()} />);

  // Настройка userEvent
  const user = userEvent.setup();

  // Заполнение формы
  await user.type(screen.getByLabelText('Email'), 'test@example.com');
  await user.type(screen.getByLabelText('Пароль'), 'password123');

  // Отправка формы
  await user.click(screen.getByRole('button', { name: 'Войти' }));

  // Проверка результатов
  expect(screen.getByText('Вход выполнен успешно')).toBeInTheDocument();
});

```

Примеры тестов

Тестирование простого компонента

```

// Button.js
import React from 'react';

function Button({ onClick, children }) {
  return (
    <button onClick={onClick}>
      {children}
    </button>
  );
}

export default Button;

// Button.test.js
import { render, screen } from '@testing-library/react';
import userEvent from '@testing-library/user-event';
import Button from './Button';

test('вызывает функцию onClick при клике', async () => {
  const handleClick = jest.fn();
  render(<Button onClick={handleClick}>Нажми меня</Button>);

  const user = userEvent.setup();
  await user.click(screen.getByText('Нажми меня'));

  expect(handleClick).toHaveBeenCalledTimes(1);
});

```

Тестирование формы

```
// LoginForm.js
import React, { useState } from 'react';

function LoginForm({ onSubmit }) {
  const [email, setEmail] = useState('');
  const [password, setPassword] = useState('');
  const [error, setError] = useState('');

  const handleSubmit = (e) => {
    e.preventDefault();
    if (!email || !password) {
      setError('Пожалуйста, заполните все поля');
      return;
    }
    setError('');
    onSubmit({ email, password });
  };

  return (
    <form onSubmit={handleSubmit}>
      {error && <div role="alert">{error}</div>}
      <div>
        <label htmlFor="email">Email</label>
        <input
          id="email"
          type="email"
          value={email}
          onChange={(e) => setEmail(e.target.value)}
        />
      </div>
      <div>
        <label htmlFor="password">Пароль</label>
        <input
          id="password"
          type="password"
          value={password}
          onChange={(e) => setPassword(e.target.value)}
        />
      </div>
      <button type="submit">Войти</button>
    </form>
  );
}

export default LoginForm;

// LoginForm.test.js
import { render, screen } from '@testing-library/react';
import userEvent from '@testing-library/user-event';
```

```

import LoginForm from './LoginForm';

test('отображает ошибку при отправке пустой формы', async () => {
  render(<LoginForm onSubmit={jest.fn()} />);

  const user = userEvent.setup();
  await user.click(screen.getByRole('button', { name: 'Войти' }));

  expect(screen.getByRole('alert')).toHaveTextContent('Пожалуйста, заполните все поля');
});

test('вызывает onSubmit с правильными данными', async () => {
  const handleSubmit = jest.fn();
  render(<LoginForm onSubmit={handleSubmit} />);

  const user = userEvent.setup();
  await user.type(screen.getByLabelText('Email'), 'test@example.com');
  await user.type(screen.getByLabelText('Пароль'), 'password123');
  await user.click(screen.getByRole('button', { name: 'Войти' }));

  expect(handleSubmit).toHaveBeenCalledWith({
    email: 'test@example.com',
    password: 'password123'
  });
});

```

Тестирование асинхронного компонента

```

// UserProfile.js
import React, { useState, useEffect } from 'react';

function UserProfile({ userId }) {
  const [user, setUser] = useState(null);
  const [loading, setLoading] = useState(true);
  const [error, setError] = useState(null);

  useEffect(() => {
    async function fetchUser() {
      try {
        setLoading(true);
        const response = await fetch(`/api/users/${userId}`);
        if (!response.ok) throw new Error('Не удалось загрузить пользователя');
        const data = await response.json();
        setUser(data);
        setError(null);
      } catch (err) {
        setError(err.message);
      } finally {
        setLoading(false);
      }
    }
    fetchUser();
  }, [userId]);
}

```

```

    }
  }

  fetchUser();
}, [userId]);

if (loading) return <div>Загрузка...</div>;
if (error) return <div role="alert">{error}</div>;
if (!user) return null;

return (
  <div>
    <h1>{user.name}</h1>
    <p>Email: {user.email}</p>
  </div>
);
}

export default UserProfile;

// UserProfile.test.js
import { render, screen, waitForElementToBeRemoved } from '@testing-
library/react';
import UserProfile from './UserProfile';

// Мокаем fetch
global.fetch = jest.fn();

test('отображает данные пользователя после загрузки', async () => {
  // Настраиваем мок fetch
  fetch.mockResolvedValueOnce({
    ok: true,
    json: async () => ({ id: 1, name: 'Иван Иванов', email: 'ivan@example.com'
  })
});

  render(<UserProfile userId={1} />);

  // Проверяем, что отображается индикатор загрузки
  expect(screen.getByText('Загрузка...')).toBeInTheDocument();

  // Ждем, пока индикатор загрузки исчезнет
  await waitForElementToBeRemoved(() => screen.queryByText('Загрузка...'));

  // Проверяем, что данные пользователя отображаются
  expect(screen.getByText('Иван Иванов')).toBeInTheDocument();
  expect(screen.getByText('Email: ivan@example.com')).toBeInTheDocument();
});

test('отображает ошибку при неудачной загрузке', async () => {
  // Настраиваем мок fetch для имитации ошибки
  fetch.mockResolvedValueOnce({
    ok: false

```

```
});

render(<UserProfile userId={1} />);

// Ждем, пока индикатор загрузки исчезнет
await waitForElementToBeRemoved(() => screen.queryByText('Загрузка...'));

// Проверяем, что отображается сообщение об ошибке
expect(screen.getByRole('alert')).toContainText('Не удалось загрузить пользователя');
});
```

Тестирование с маршрутизацией

Для тестирования компонентов, которые используют React Router, можно использовать `MemoryRouter` для создания изолированного окружения маршрутизации.

```
import { render, screen } from '@testing-library/react';
import { MemoryRouter, Routes, Route } from 'react-router-dom';
import Home from './Home';
import About from './About';
import NotFound from './NotFound';
import App from './App';

test('отображает компонент Home на корневом маршруте', () => {
  render(
    <MemoryRouter initialEntries={['/']}>
      <Routes>
        <Route path="/" element={<Home />} />
        <Route path="/about" element={<About />} />
        <Route path="*" element={<NotFound />} />
      </Routes>
    </MemoryRouter>
  );

  expect(screen.getByText('Домашняя страница')).toBeInTheDocument();
});

test('отображает компонент About на маршруте /about', () => {
  render(
    <MemoryRouter initialEntries={['/about']}>
      <Routes>
        <Route path="/" element={<Home />} />
        <Route path="/about" element={<About />} />
        <Route path="*" element={<NotFound />} />
      </Routes>
    </MemoryRouter>
  );

  expect(screen.getByText('О нас')).toBeInTheDocument();
});
```

```
});

test('отображает компонент NotFound для неизвестного маршрута', () => {
  render(
    <MemoryRouter initialEntries={['/unknown']}>
      <Routes>
        <Route path="/" element={<Home />} />
        <Route path="/about" element={<About />} />
        <Route path="*" element={<NotFound />} />
      </Routes>
    </MemoryRouter>
  );

  expect(screen.getByText('Страница не найдена')).toBeInTheDocument();
});
```

Тестирование с Redux

Для тестирования компонентов, которые используют Redux, можно обернуть компонент в **Provider** с тестовым хранилищем.

```
import { render, screen } from '@testing-library/react';
import { Provider } from 'react-redux';
import { configureStore } from '@reduxjs/toolkit';
import userReducer from './userSlice';
import UserProfile from './UserProfile';

test('отображает информацию о пользователе из хранилища Redux', () => {
  // Создаем тестовое хранилище с начальным состоянием
  const store = configureStore({
    reducer: {
      user: userReducer
    },
    preloadedState: {
      user: {
        data: { id: 1, name: 'Иван Иванов', email: 'ivan@example.com' },
        loading: false,
        error: null
      }
    }
  });

  render(
    <Provider store={store}>
      <UserProfile />
    </Provider>
  );

  expect(screen.getByText('Иван Иванов')).toBeInTheDocument();
});
```



```
expect(screen.getByText('Email: ivan@example.com')).toBeInTheDocument();
});
```

Лучшие практики

1. **Тестируйте поведение, а не реализацию:** Фокусируйтесь на том, что пользователь видит и с чем взаимодействует, а не на внутренней реализации компонентов.
2. **Используйте доступные запросы:** Предпочитайте запросы, которые отражают то, как пользователь находит элементы на странице. Порядок предпочтения: **ByRole**, **ByLabelText**, **ByPlaceholderText**, **ByText**, **ByDisplayValue**, **ByAltText**, **ByTitle**, **ByTestId**.
3. **Используйте **userEvent** вместо **fireEvent**:** **userEvent** предоставляет более реалистичную симуляцию пользовательских взаимодействий.
4. **Избегайте тестирования реализации:** Не тестируйте состояние компонента или его методы напрямую. Вместо этого тестируйте результаты действий пользователя.
5. **Используйте **data-testid** как последнее средство:** Если нет другого способа выбрать элемент, используйте атрибут **data-testid**.
6. **Тестируйте доступность:** Используйте запросы **ByRole** для проверки доступности вашего приложения.
7. **Изолируйте тесты:** Каждый тест должен быть независимым от других тестов.

Заключение

React Testing Library предоставляет мощный и интуитивно понятный способ тестирования React-компонентов, фокусируясь на поведении пользователя, а не на внутренней реализации. Это делает тесты более надежными и менее хрупкими при рефакторинге.

Используя RTL, вы можете быть уверены, что ваши тесты проверяют то, что действительно важно для пользователей вашего приложения, а не детали реализации, которые могут измениться со временем.